# DefenSYS: system audit and feature flows

Code-derived review for owner verification • 16 October 2026 • Asia/Manila

## 1. Scope and how to review

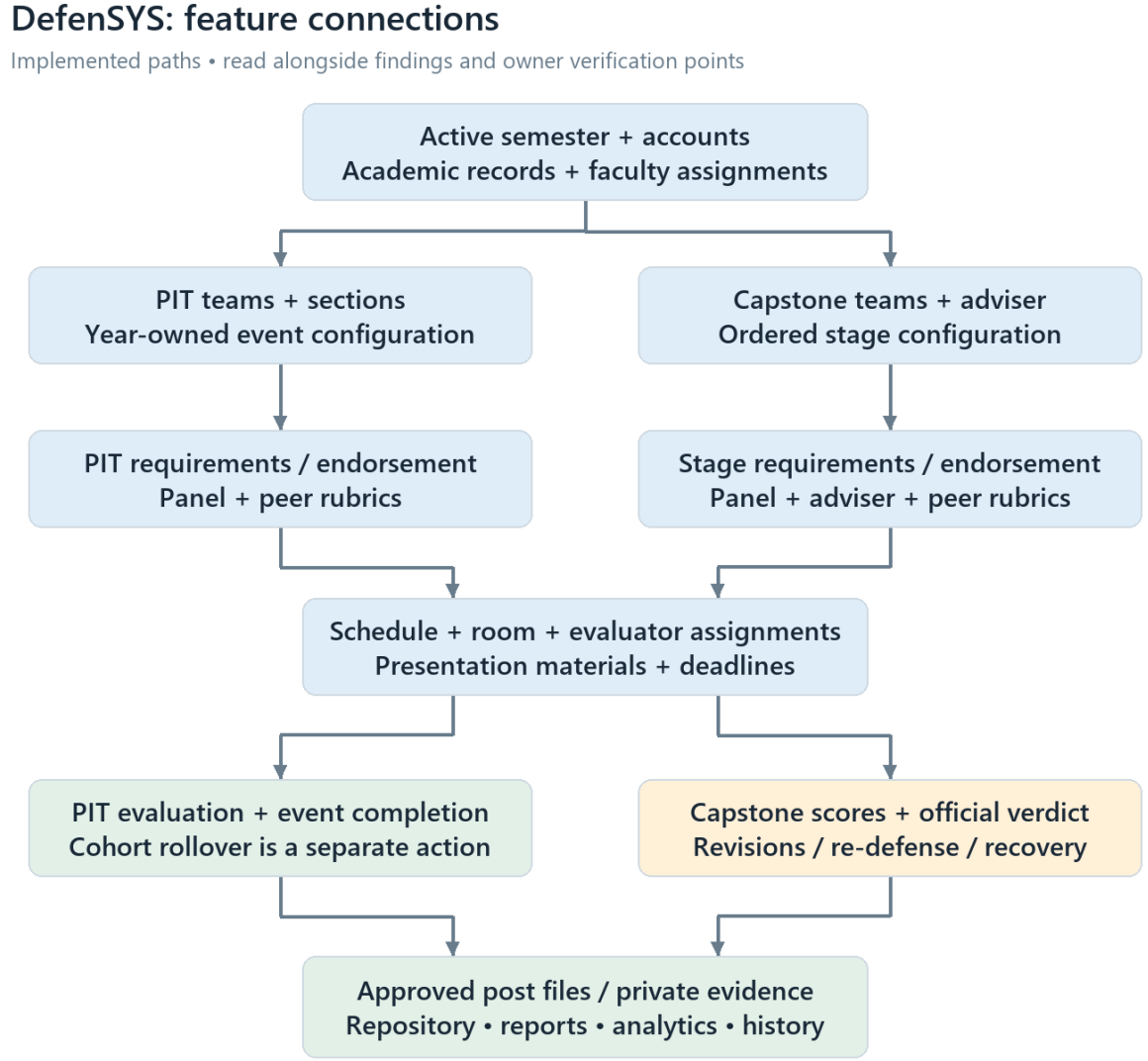
This maps the current local working tree at commit 21942de, including the uncommitted changes present when the audit began. It describes implemented behavior. It does not assume that existing product documents, comments, or screen labels accurately describe the code.

Coverage: all 12 top-level backend feature directories; 49 Django model classes; all 181 resolved /api/ URL patterns, including aliases and report catalog expansion; Flutter navigation, feature providers, and principal role workspaces. The technical appendix lists every resolved API route and model. A route existing does not establish that its operation is available to a particular role.

Evidence labels: IMPLEMENTED means a code path exists; TESTED means the named focused checks passed; REVIEW means an institutional rule needs your decision; FINDING means the code contains a concrete inconsistency or risk. This is a functional, architecture, and permissions audit with targeted runtime tests. It is not a complete penetration test, production environment inspection, or click-through of every screen.

Use the F identifiers to discuss a feature, A identifiers to discuss an audit finding, and V identifiers to confirm an institutional rule. Suggested response: “V08 correct; V12 should allow adviser grading before scheduling; A03 should follow the chair verdict.” No business rules were changed by this audit.

# 2. Whole-system connection map




The active semester is the common context. Student academic records establish the term, year, and section. Faculty assignments establish responsibilities. Teams join students, an adviser or section instructor, project identity, and the term. PIT event configurations and Capstone stage configurations define requirements and grading. Schedules join a team to a stage/event, evaluators, room, time, rubric, and optionally a documenter. Submissions, grades, verdicts, and signed minutes then feed progression, archive views, dashboards, reports, and analytics.

The core connection key is more specific than a team ID: team + semester + PIT/Capstone scope + stage/event + project version + defense attempt where applicable. Different modules enforce different parts of this key. Several findings below concern places where one part is missing.

Architecture: Flutter web/mobile → Riverpod providers → shared authenticated HTTP client → Django REST views → permission and row scopes → domain services → PostgreSQL and file storage. Django Channels and Redis carry selected change notifications to clients; clients refresh their data. JWT authenticates institutional users; guest JWTs use a separate evaluator route. Protected media is served through /api/media/files/ and feature-specific file endpoints. Deployment templates include reverse proxy and service configuration; deployment state was not inspected.

# 3. Roles, responsibilities, and entry points

Base account roles are admin, faculty, and student. Faculty capabilities can overlap; the system also derives access from actual adviser, instructor, panel, and documenter assignments. External evaluators receive a guest identity rather than a normal faculty account.

Admin - Web administration: academic terms, accounts/imports, academic records, team management, Capstone stages, rubrics, scheduling, Evaluation & Grades, Project Archive, curriculum analytics, Audit & Compliance. Admin can review nominations and external evaluators, authorize recovery, and use protected correction workflows. Some actions, such as signing minutes, explicitly require the assigned signatory even if the caller is an admin.

PIT lead - Faculty web workspace scoped to an assigned PIT year: cohort roster/imports, section instructors, PIT teams, PIT events, rubrics, scheduling, grades, and archive/audit. Third-year PIT enters a historical/audit mode during the Capstone intake term. Some older document/report endpoints do not enforce this scope consistently; see A01 and A02.

PIT section instructor - Assigned by semester + year + section. Uses the faculty workspace for assigned PIT teams and deliverables. This is an assignment record, not another base account role. Instructor access does not automatically grant PIT event or schedule management.

Capstone adviser - Assigned team oversight, requirement review and endorsement, adviser assessment, weekly progress review, revision clearance, and adviser signature on minutes. The is_adviser capability and StudentTeam.adviser assignment are separate; both matter in relevant workflows.

Panelist / panel chair - Assigned defense evaluations, draft scores, final submissions, and results. The chair additionally records official Capstone verdicts and signs minutes. Panel eligibility is distinct from assignment to a particular schedule. A team's adviser cannot also sit on that team's panel.

Documenter - Assigned Capstone minutes, comments for institutional and external panelists, documenter signature, and retained signed records. Documenter workspace exists on mobile and web. The signed minutes workflow can notify the adviser and documenter.

Uploader - Faculty document workspace for general TeamDocument uploads. It is separate from stage deliverables and from public repository publication. Legacy direct archive upload routes remain registered but current repository scope disables those uploads for every supported archive management role.

Student / team leader - Student workspace on web or mobile, team and event context, stage deliverables, peer evaluation when available, repository browsing, notifications, and profile/settings. Weekly progress submission is specifically limited to Capstone team leaders. Some UI copy and older documents incorrectly imply students are mobile-only.

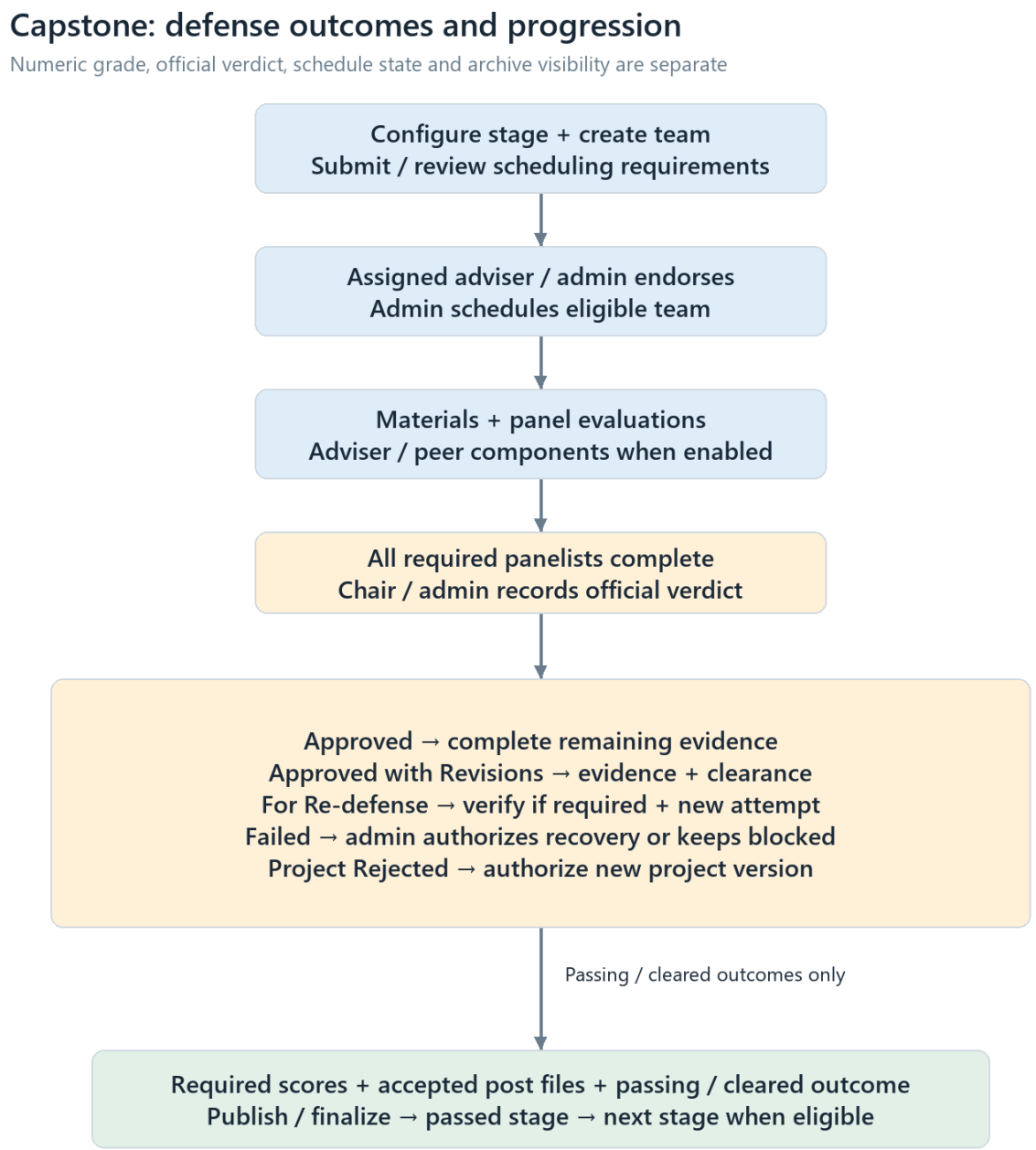
Section project manager - Student assigned to a SectionAssignment. Gets section integration context and broader section team/schedule visibility. This is an assignment, not a base role; it does not grant faculty grading or administration powers.

External evaluator - Approved ExternalEvaluator record → invitation for specified schedules → code/link exchange → restricted guest evaluation workspace → draft/final scores and own results. Guest access expiry/revocation does not erase the historical assignment or minutes comments. Guests do not issue the institutional chair's verdict.

Routing evidence: frontend/lib/navigation/workspace_access.dart, app_router.dart, admin_route_paths.dart. Authorization evidence: authentication_access_control/scopes.py; user_management/permissions.py; feature-specific services and views. Frontend navigation guards help users reach the right workspace; backend checks determine actual authority.

# 4. Detailed lifecycle flows

## Flow C: Capstone from intake to progression




C01. Admin configures the academic term, enables team creation when appropriate, imports students and creates academic records. The calendar derives Capstone 1 from second semester and Capstone 2 from first semester. Creation is also controlled by the stored team-creation flag; current behavior permits new teams in continuation terms when enabled.

C02. Admin forms/imports a Capstone team with leader, members, project title, term and adviser. The project starts at version 1. Stages are dynamically configured and ordered; the implementation does not require a fixed hardcoded list such as Concept → Proposal → Final.

C03. Configure each active stage: deliverables or presentation-only mode, grading weights, published rubrics for required evaluator types, and optional authoritative minutes. Students upload the before-endorsement requirements. Assigned adviser/admin reviews them; rejection returns the file with feedback. Review-and-endorse accepts the displayed scheduling files and endorses atomically, using a signed review snapshot to reject stale

reviews.

C04. Scheduling requires the appropriate endorsement/readiness and completion of earlier active stages. Admin generates a plan or creates a schedule, reviews it, then confirms. The schedule fixes team, project version, stage, rubric, panel/chair, date, room, duration and optional documenter/external evaluators. Conflicts are checked for room and shared evaluators/documenter, including overlapping dates.

C05. Presentation materials use a separate before-defense deadline. These are not necessarily the files that gate endorsement. A configured late-upload window only accepts missing files and preserves existing evaluated submissions. Accepted or finalized files have additional lock rules.

C06. On or after the defense date, assigned panelists and external evaluators submit complete evaluations. Availability is day-based; it is not limited to the exact scheduled clock slot. Drafts can be saved; changed rubric/roster/schedule context invalidates stale drafts. The target can be team, individual, or both. Every required evaluator must finish before the chair's verdict.

C07. Capstone peer evaluation opens after the scheduled date is reached and all required panel evaluations have been submitted, subject to the term switch and component weight. Each member evaluates every other member: N × (N − 1) submissions; self-evaluation is excluded. Adviser grading requires the current endorsed/scheduled context, assigned adviser, enabled term policy, and nonzero weight.

C08. The chair/admin records one official outcome: Approved; Approved with Revisions; For Re-defense; Failed; Project Rejected. These outcomes are separate from the numeric grade and schedule status.

C09. Approved with Revisions → provide requested evidence/post-defense files → chair, assigned adviser, or admin records clearance. A compliance review date can be recorded. A passing outcome with unresolved revisions does not count as passed-and-cleared.

C10. For Re-defense → adviser verification if explicitly required → create another schedule/attempt → retain prior assessment history → evaluate again. Failed → admin records institutional recovery decision: authorize retake, authorize replacement concept, or keep blocked. Project Rejected requires a replacement concept rather than a retake of that concept. A replacement increments project version, resets stage context to the first active stage, and preserves old project evidence as history.

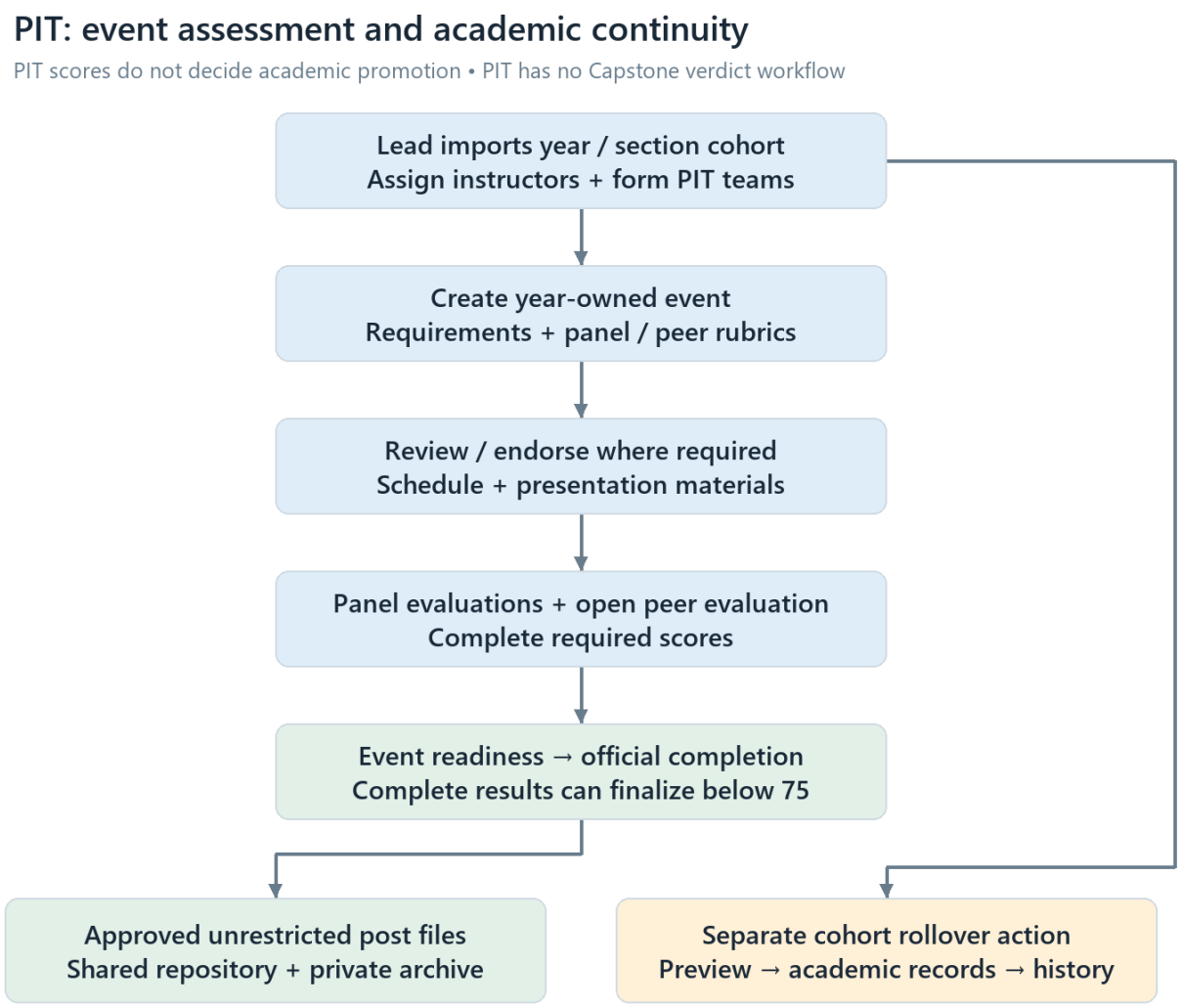
C11. Complete required grade components, accepted post-defense requirements, and passing/clearance rules. Stage/group official completion and per-team publication are distinct. Eligible records can auto-finalize after official completion; explicit publication also exists. A published passing-and-cleared stage with accepted required post files becomes passed. All earlier active stages must be passed/archived and have their required post files accepted before the next stage is ready.

C12. Signed minutes are a system record and cannot be fulfilled by uploading a student PDF. They appear separately in requirements/private archive. Current post_deliverables_complete checks only required type=post submissions, so minutes are not included in that particular progression gate; verify whether the institution wants them to be mandatory before progression.

C13. Approved nonrestricted post-defense submissions feed repository browsing. Private archive/audit also retains restricted evidence and signed minutes. Direct upload queue eligibility has a different numeric passing check from verdict-based completion; see A04. Student academic rollover and active-semester switching are separate administrative actions, not automatic consequences of finishing a defense.

Source anchors: student_teams/services.py; defense/scheduler/panelist_evaluation.py, sessions.py, progress.py; grading/grades/availability.py, defense_workflow.py, services.py; repository/deliverables/endorsement.py, timing.py, services.py; defense/minutes/requirements.py.

## Flow P: PIT from cohort setup to history




P01. Configure term → import the PIT cohort or official class list → create StudentAcademicRecord rows for the assigned year/section → assign section instructors → form/import PIT teams. A PIT lead is assigned a year; section instructors are assigned specific sections in a semester.

P02. PIT lead/admin creates a PitEventGradingConfig owned by term + year with an event name/code, panel and peer rubrics, weights, peer opening policy, deliverables, and archive naming template. PIT has no adviser component or official Capstone verdict workflow. Default weights are panel 80% and peer 20%, configurable to total 100%.

P03. Students prepare configured requirements → faculty review/endorsement when the event has scheduling requirements → lead/admin schedules the event. An event without before-endorsement requirements does not need a fabricated endorsement step. Before-defense material deadlines still apply if configured.

P04. Assigned institutional/external panelists evaluate → students complete peer assessment when the event switch opens it → system calculates per-team/per-student results as the rubric requires. Complete PIT scores can finalize even below 75. The score indicator is not an academic promotion decision.

P05. Lead/admin checks event-wide readiness → officially completes the event → records finalize and become historical assessment evidence. Approved unrestricted post files feed the repository. Archive/replacement actions remain scoped to the PIT year.

P06. PIT cohort rollover preview determines source and target term/year, marks existing destination records, and blocks missing academic periods. Confirm creates target academic records, preserves section and rolled_from links, and skips duplicates. It does not automatically promote based on PIT defense scores, move whole teams into

Capstone, or activate the next term. Admin separately forms Capstone teams at intake. Third-year PIT management becomes historical/audit-only during the matching intake context.

REVIEW: PIT event names are unique per semester, even though events are year-owned. The same event name cannot be reused independently by first-year and second-year cohorts in that term without a naming distinction.

Source anchors: dashboards/views.py:_pit_rollover_preview_payload and PitLeadCohortRolloverConfirmView; defense/scheduler/event_scope.py and models.py; student_teams/term_scope.py; grading/grades/services.py; user_management/views.py.

### Flow M: minutes and signatures

Assigned Capstone schedule → documenter opens minutes → snapshot team/project/date/room/roster → record comments for every current faculty and external evaluator → save draft → documenter signs and submits → assigned adviser signs → assigned chair signs → generate final PDF → expose through authorized minutes/private archive routes. Draft preview is distinct from the final signed PDF. Retained revisions preserve earlier signed evidence. An e-signature image is required for signing. PIT schedules do not use this minutes workflow.

### Flow R: file approval, repository, and replacement

Stage/event definition → student uploads real file → DeliverableSubmission plus file-history rows → assigned faculty accepts/rejects → accepted nonrestricted post file appears in the authenticated public-facing repository → searchable metadata and PDF extraction/classification support discovery → personal bookshelf and reviews attach to the browsable entry.

Accepted archive file needs correction → authorized archive manager records resubmission/replacement reason → replacement flag opens the specified file → authorized submitter replaces it → new version enters review → acceptance locks it again. Cancellation and trail/export actions exist. General TeamDocument upload is a separate path and does not, by itself, satisfy a configured stage requirement or publish a repository entry.

“Public repository” in code means the shared browsing surface for authenticated users. The /api/repository/archive/ endpoints require authentication; it is not an anonymous internet library.

### Flow T: academic transition

Review current outstanding schedules, assessments and archive obligations → preview transition blockers → resolve them or admin supplies a forced-switch reason → activate target semester transactionally → deactivate previous semester → write SemesterTransitionLog and system audit evidence. Student academic-record rollover and Capstone team continuation are separate from activation. Historical teams are normally read-only, although legacy endpoints need additional consistency work.

## 5. Feature-by-feature inventory

Each entry gives the actor, implemented action chain, output, main connections, and source. Screen labels may differ from internal names such as grade_center or archive.

### F01. Login, sessions, logout, and workspace switching

All institutional users: credentials → throttled login → access/refresh JWT and user profile → terms gate where required → role/workspace routing. Shared HTTP client adds Bearer token, coordinates refresh, retries supported failures, and expires invalid sessions. Browser session/local storage and mobile secure storage support different persistence policies; remember-me changes refresh policy. Defaults in settings are 8-hour access, 12-hour standard refresh, 7-day remembered refresh, overridable by environment. Guest sessions use their own exchange path. Connections: every authenticated feature, realtime reconnect, notifications. Sources: authentication_access_control/views.py and serializers.py; frontend/services/auth, network, session; navigation/workspace_access.dart.

## F02. Profile, credentials, and recovery

User edits permitted profile/avatar fields or changes password. Forgotten password: identifier → email/SMS OTP → verify within expiry/attempt limits → reset token → validated new password → notification email. Admin reset is a separate explicit administrative API. Faculty upload/delete an e-signature for minutes/reports. Sources: authentication_access_control/password_reset.py; user_management/views.py:UserESignatureView and AdminResetPasswordView. Finding A05 covers predictable password paths; no passwords were changed in this audit.

## F03. Academic periods and active-term safety

Admin creates school year/semester → configures term policy → previews impact → activates term or uses reasoned forced transition. Exactly one active semester is enforced by model/constraint. Outputs: term context used across rosters, teams, grades, schedules, dashboards and history. Source: academic_period_management/models.py, services.py, views.py, capstone_mode.py.

## F04. Account directory and bulk enrollment

Admin creates/edits/deactivates accounts or imports a batch. Student imports establish term/year/section context separately from account identity; faculty/general imports use their own context. Validation handles duplicate/conflicting records and reports outcomes. PIT leads have dedicated student-only/import routes constrained to their cohort. Sources: user_management/views.py and frontend/admin/user_management/bulk_import.

## F05. Student academic records and admin rollover

Admin manages student + semester + year + section rows → previews promote/retain/drop/create actions → confirms destination records → preserves rolled_from references and skips duplicates. Current admin confirm also invokes a broad Capstone team advancement helper; see A07. Connections: team eligibility, cohort roster, section ownership, continuation. Source: user_management/academic_records/models.py, rollover.py, views.py.

## F06. Faculty work roles and histories

Admin grants faculty capability/assignment → active responsibilities unlock workspace/actions → record FacultyRoleAssignment history. Assigning is_adviser does not choose a team's adviser. Team adviser changes have their own TeamAdviserAssignment history and conflict checks. Panel/documenter assignments remain schedule-specific. Sources: user_management/role_assignments.py, models.py, views.py; student_teams/adviser_assignments.py.

## F07. Panelist nomination and eligibility

PIT lead/admin nominates faculty with a reason → pending PanelistEligibilityRequest → admin approves/rejects → eligibility changes → eligible faculty can be selected for schedules. Existing eligibility and duplicate requests are handled. Eligibility does not itself assign a schedule or make someone the chair. Sources: user_management/panelist_eligibility.py and panelist_eligibility_views.py.

## F08. External evaluators and guest access

Lead/admin registers evaluator → admin approval → authorized manager selects eligible schedules → creates expiring/revocable invitation → guest exchanges code/link for guest JWT → sees only assigned defenses → drafts/submits evaluation → views own results. External identities also contribute to evaluator-completion checks and minutes roster. Sources: user_management/external_evaluators.py, external_evaluator_views.py, models.py; defense/scheduler guest views; frontend/screens/guest.

## F09. PIT cohort, official class list, and cohort rollover

PIT lead opens cohort → imports student batch/official class list → resolves roster/section/team membership → inspects active/history tabs → previews and confirms record rollover. This is not whole-team carryover and is not a score-based promotion engine. Sources: dashboards/views.py; user_management/views.py:PitLeadOfficialClassListImportView; frontend/faculty/pit_lead.

## F10. Section instructors and project managers

Lead/admin assigns faculty to term/year/section → instructor sees assigned PIT team work. SectionAssignment separately identifies a student project manager and section integration/system context. Neither record makes the person a panelist. Sources: user_management/models.py:SectionInstructorAssignment; student_teams/models.py:SectionAssignment; user_management/views.py; frontend/student/section_integration_tab.dart.

## F11. Team creation, bulk import, detail, and adviser assignment

Admin/scoped PIT manager chooses valid term/year/section → selects eligible students and leader → creates/imports team → assigns adviser where appropriate → tracks members, project title/version and stage state. Import preview precedes import; active/history context controls edits. Historical data is not supposed to become a current team simply because an ID is known. Sources: student_teams/views.py, serializers.py, term_scope.py, team_levels.py; frontend/admin/student_teams and team_detail.

## F12. General team documents and uploader workspace

Authorized user/uploader chooses team → uploads TeamDocument with document type, file bytes and metadata → team documents can be listed/downloaded/deleted under their own rules. This path does not create DeliverableSubmission approval/endorsement state. Uploader can access active-term teams; PIT lead access is currently much broader than the shared scope policy (A01). Sources: student_teams/documents/views.py and models.py; frontend/web/uploader/uploader_dashboard.dart.

## F13. Weekly Capstone progress

Capstone team leader submits week/file/report → own reports appear in student view → assigned adviser reviews available reports → authorized compilation can combine weekly reports into an output. Submission chooses a team through a first-membership lookup, not an explicit active-term selector. Detail/list/file permission rules differ (A02). Sources: student_teams/weekly_progress/views.py; repository/deliverables/views.py:CompileWeeklyReportsView; frontend/faculty weekly progress screen.

## F14. Capstone stage setup and ordering

Admin creates/edits/reorders/activates stages → declares presentation-only or file requirements, minutes settings, labels/codes and archive behavior → attaches per-term grading config → setup readiness checks weights and published compatible rubrics. Stage definitions and deliverables are global, while StageGradingConfig is term-specific. Historical checklists therefore do not have a completely separate term-versioned definition. Sources: defense/stages/models.py, views.py, readiness.py, grading_config.py.

## F15. PIT event setup

Lead/admin creates year-owned event in a term → chooses panel/peer rubrics and weights → configures requirements/deadlines/format/restriction/naming → opens peer evaluation or completes event under readiness rules. An existing classified event cannot move to another year. Source: defense/scheduler/models.py:PitEventGradingConfig and PitEventDeliverable; views.py:PitEventConfigLookupView; event_scope.py; frontend/faculty/events.

## F16. Rubric engine

Admin/scoped manager creates draft rubric → sets PIT/Capstone scope, panel/adviser/peer evaluator type, team/individual/both targets, criteria, maximum scores, weights/scales → validates → publishes → assigns to stage/event/schedule. Scoring validates the exact criteria and permitted students, rather than trusting client totals. Sources: grading/rubrics/models.py, views.py; grading/grades/scoring.py; defense/scheduler/panelist_evaluation.py; frontend/admin/rubric_engine.

## F17. Stage deliverables, feedback, and file history

Configured definition → permitted team member uploads → server validates file/timing/context → extraction/classification for suitable PDFs → submitted item → faculty accepts/rejects with feedback → student corrects rejected item. Definition includes required/optional, pre/post, scheduling/before-defense timing, format, archive template, restriction and panel-material visibility. Current submission and historical uploaded files are separate records. Sources: repository/deliverables/models.py, services.py, views.py, pdf_processor.py.

## F18. Review-and-endorse and cancel endorsement

Faculty/admin loads required scheduling files and review token → reviews current file snapshot → atomically accepts and endorses → team becomes eligible for schedule selection. File/checklist changes cause a stale-review conflict. Missing/rejected files block endorsement. Cancel endorsement is blocked after an active/done schedule exists. Capstone requires the assigned adviser/admin; PIT uses its scoped responsible faculty path. Source: repository/deliverables/endorsement.py; services.py:endorse_team/unendorse_team.

## F19. Presentation materials and deadline controls

Schedule establishes defense date/time → required before-defense material deadline is derived → student uploads before cutoff → authorized manager can extend under supported conditions or open a missing-file late window with reason → panel sees configured material. Finalized/historical evaluations lock materials; late window does not replace files already evaluated. Sources: repository/deliverables/timing.py; defense/scheduler/materials.py; views.py:DefenseMaterialDeadlineView.

## F20. Scheduling, plan generation, and confirmation

Manager selects term/scope/stage or PIT event and ready teams → chooses panel/chair/documenter/external evaluators, room, dates, duration → generates plan → checks conflicts/readiness → confirms transactionally. Batch/session identifiers group rows. Manual list/create/detail and board import UI coexist with plan APIs; UI import is not an extra unlisted backend route. Sources: defense/scheduler/serializers.py, sessions.py, views.py; dashboards schedule-planner; frontend/admin/defense_scheduler.

## F21. Defense board and operational changes

Board reads schedules and derived workflow states → authorized manager previews updates/deletion → supplies expected revisions → applies selected schedule/session/stage/semester operation → audit evidence records reason and changes. Paused/postponed/no-show are operational states distinct from scheduled/done/cancelled/archived. Deletion is blocked by grades, authored drafts, comments, signatures, completed records or correction history. Sources: defense/board/views.py; defense/scheduler/operations.py, services.py, progress.py.

## F22. Panel evaluation drafts, submission, and results

Assigned evaluator opens defense on/after date → views rubric/materials → drafts team/individual/both scores → saves matching context → submits a complete criterion set atomically → stored submissions/criterion rows recompute panel and student scores → evaluator results retain previous attempts. Guest versions use guest authentication and assigned invitation scope. Sources: defense/scheduler/views.py and panelist_evaluation.py; grading/grades/services.py, scoring.py; frontend/app/panelist.

## F23. Peer evaluation

Availability policy opens the current grade context → each member rates every other member against the configured peer rubric → validate team/target/criteria → store one evaluator/evaluatee assessment → aggregate student and team peer components → completion count feeds grade readiness. Capstone waits for all required panelists; PIT uses its explicit event opening switch. Manual peer override can mark the component complete even without all raw submissions; verify whether this administrative capability is intended. Sources: grading/grades/peer_eval.py, peer_views.py, availability.py.

## F24. Adviser grading

Assigned Capstone adviser opens eligible current defense → receives the configured adviser rubric → submits allowed criterion scores → aggregates adviser component → recalculates grade. Disabled policy/zero weight changes availability, while required-component and stored numeric-completeness logic are separate. PIT does not use adviser grading. Sources: grading/grades/adviser_views.py, availability.py, services.py; frontend/services/grading/adviser_grading_provider.dart.

## F25. Numeric aggregation, grade center, and official completion

Panel/adviser/peer evidence → normalize rubric percentages → apply configured component weights → team and per-student stage records → readiness shows missing evaluators/components → manager reviews group readiness → officially completes stage/event → eligible records publish/finalize. Capstone default composition is 50/30/20; PIT default is 80/20. Passing numeric threshold is 75, but explicit Capstone verdicts override numeric result in core workflow. Publication locks routine edits. Complete PIT scores can publish below 75. Sources: grading/constants.py; grades/models.py, scoring.py, services.py, views.py; frontend/admin/grade_center.

## F26. Official verdict, revisions, and re-defense

All required panel evaluations complete → chair/admin records outcome → approved flows toward completion; revisions flow to clearance; re-defense flows to next attempt; failed/rejected flows to admin recovery. This is Capstone-only. Time elapsing or entering a score alone does not prove completion. Sources: grading/grades/defense_workflow.py; defense/scheduler/progress.py; student_teams/services.py.

## F27. Recovery and replacement concepts

Failed/rejected assessment → admin records decision/reason → retake authorization or new-concept authorization → next attempt consumes applicable authorization. New concept increments project version, creates first-stage context and preserves old schedules/files/grades through all_objects/history paths. Old project assessments are read-only. Source: grading/grades/defense_workflow.py; student_teams/project_versions.py; models.py:TeamRecoveryAuthorization.

## F28. Grade corrections and protected amendments

Admin requests correction with reason, desired changes and expected updated timestamp → preview computes before/after inside rolled-back savepoint → ordinary permitted correction applies, or protected correction remains pending → admin approves/rejects with explicit acknowledgment → stale snapshot blocks apply → preserve GradeCorrection and audit evidence. Do not assume two different admins are required: the review function does not prohibit requester self-approval. Sources: grading/grades/corrections.py, correction_models.py, views.py:GradeCorrectionsView.

## F29. Defense minutes and signed PDF

Documenter draft/comments → documenter signature → assigned adviser signature → assigned chair signature → completed PDF → authorized preview/download and private archive references. Current and former panel comments/retained revisions preserve evidence. Configured system minutes cannot be replaced through normal deliverable review/upload. Sources: defense/minutes/models.py, views.py, services.py, requirements.py; frontend/documenter workspaces.

## F30. Project Archive management and audit

Admin/PIT lead opens scoped archive → browse stage/team/folder evidence, missing requirements, restricted files, signed minutes and history → request resubmission/cancel replacement → authorized replacement → trail/export. /repository/audit/ aliases /repository/project-archive/. Legacy upload-pit/upload-capstone routes exist, but repository_scope currently returns can_upload=false. Sources: repository/project_archive/services.py, grouping.py, folders.py, payloads.py, trail.py and views.py.

## F31. Shared project library, search, bookshelf, and reviews

Authenticated user browses approved/unrestricted post deliverables and approved legacy PIT entries → groups project outputs → previews actual abstract/background or document excerpt → keyword-ranked search/suggestions across metadata/content → saves bookshelf entry → adds/updates/removes own review. Pre-defense submissions and restricted outputs do not belong in this shared list. Restriction matching is currently by globally gathered deliverable IDs (A06). Sources: repository/archive/services.py, library.py, ml_search.py, views.py, models.py; frontend/student/repository.

## F32. PDF text extraction and classification

Suitable uploaded PDF → extract text, topics and summary metadata → Naive Bayes classification/status/confidence/evidence → persist fields/cache → search and analytics consume the result. Library abstract/background excerpts use deterministic section extraction rather than generated research conclusions. This is document indexing/classification, not an LLM chatbot, plagiarism detector, or automatic defense verdict engine. Sources: repository/deliverables/pdf_processor.py and classifier-related files; repository/archive/ml_indexing.py, library.py, ml_search.py.

## F33. Dashboards, reports, and curriculum analytics

Role-scoped data → admin/faculty/student/panelist dashboards and planner/cohort views. Report catalog includes academic periods, enrollment, faculty, rubric/setup, PIT setup, readiness, overview, project records, external evaluators and curriculum summary; dedicated reports include team/individual/semester grades, schedules, rosters, directories and audit trail. Export dispatcher supports preview/JSON, PDF, CSV, XLSX and Word-style exports, varying by report. Curriculum analytics/explorer is admin-only through service checks: project/domain/technology evidence, rubric competencies, evaluator calibration, funnel/longitudinal/remediation analysis, scenarios and proposals. Its proposal POST computes content; it does not implement an institutional curriculum approval workflow. Sources: dashboards/views.py; reports/catalog_reports.py, views.py, faculty_reports.py, export_formatters.py; curriculum_analytics/services.py, explorer.py, views.py.

### F34. Notifications, realtime, audit review, and history

Domain action → recipient/workspace-specific in-app Notification with action context → user reads/follows it or marks read/all-read. Email/SMS services support account recovery and configured notification paths; SMS defaults to console unless a gateway backend is configured. Realtime authenticates JWT, joins user/term/PIT groups, sends change events and revalidates token/account state. SystemAuditLog records high-impact before/after/actor/reason/context; authorized admin/scoped lead reviews evidence, preserving SystemAuditReview identity snapshots. Personal history and repository trails are separate read surfaces. Sources: notifications; realtime; authentication_access_control/audit.py, scopes.py, views.py. Provider-side delivery, Redis connectivity and production origin handling were not exercised.

## 6. Data and state connections

User → StudentAcademicRecord → Semester → SchoolYear is the enrollment axis. User → FacultyRoleAssignment / SectionInstructorAssignment / TeamAdviserAssignment is the responsibility/history axis. StudentTeam → TeamMembership → User is the roster axis. SectionAssignment adds the project-manager context.

Capstone: DefenseStage → StageDeliverable; DefenseStage + Semester → StageGradingConfig → panel/adviser/peer Rubric → RubricCriterion. PIT: Semester + year → PitEventGradingConfig → PitEventDeliverable and panel/peer rubrics. These are different configuration paths.

StudentTeam → DefenseSchedule → SchedulePanelist and GuestPanelistCode → ExternalEvaluator is the assessment-assignment axis. DefenseSchedule also owns draft context and optional DefenseMinutes → comments/revisions. TeamGrade links a team, term, scope, stage/event and current schedule; GradeAttemptHistory preserves past attempts when the current context moves.

TeamGrade → PanelistGradeSubmission → PanelistCriterionScore; TeamGrade → GradeBreakdown; TeamGrade → PeerEvaluationSubmission; TeamGrade → StudentStageGrade is the grading evidence axis. GradeCorrection records amendments. TeamStageProgress and TeamRecoveryAuthorization drive Capstone progression/recovery.

StudentTeam → DeliverableSubmission → DeliverableSubmissionFile is the requirement/file-history axis. ArchiveEntry represents legacy archive records. ProjectArchiveLog records archive actions. RepositoryReview and UserBookShelf attach user interaction to browsable entries. TeamDocument and WeeklyProgressReport are independent supporting-document paths.

State distinction 1: Schedule.status = scheduled/done/cancelled/archived; operation_state = normal/paused/postponed/no_show; display_status is derived from assessment evidence. “Done” can mean an attempt was assessed, including re-defense or failure, rather than that the academic stage passed.

State distinction 2: TeamGrade.status = pending/awaiting_peers/published; verdict = official Capstone outcome; result = verdict-sensitive pass/revisions/re-defense/fail; is_officially_complete = group configuration flag. These values should not be substituted for one another in UI filters or reports.

State distinction 3: TeamStageProgress = locked/ready/scheduled/grading/passed/failed/archived/revisions_pending/for_redefense. StudentTeam.status is a coarser team indicator. PIT's latest-event Approved/Failed indicator is not student promotion or an overall semester grade.

State distinction 4: DeliverableSubmission status = pending/accepted/rejected; accepted-file replacement/revision flags govern permitted edits. Requirement setup readiness, upload completeness, endorsement, presentation-material readiness, grading readiness, and archive visibility are separate checks.

## 7. Audit findings and discrepancies

These are observations from current code. Proposed changes below were not applied. Focused tests passing does not establish that these uncovered paths are safe or institutionally correct.

## A01 - High: PIT lead document access bypasses cohort boundaries

team_document_queryset_for_user returns all TeamDocument rows for any is_pit_lead user. user_can_access_team also returns true for a PIT lead without checking term, PIT scope, or assigned year. Direct document upload/detail/download operations use this helper. This differs from visible_teams_for, which limits PIT management to its assigned year. Impact: a PIT lead can reach unrelated-year or Capstone general documents through these APIs, and the same helper permits uploads outside the expected scope.

Evidence: backend/modules/student_teams/documents/views.py:16, :26, :46. Confidence: direct static control-flow finding; no exploit was run on real user data. Proposed direction: use shared row-scoped access plus operation-specific write rules; test cross-year and Capstone denial.

## A02 - High: weekly-report list, detail, and file access disagree

PIT-lead list returns all reports. Detail get_object permits any faculty account that is not flagged adviser to read a known report ID, because only the adviser branch checks team assignment. The dedicated file endpoint is narrower. Submission also selects the first membership across terms rather than an explicit current Capstone team. Impact: report metadata/content can be exposed beyond assignments, and students with historical memberships can target an unintended team.

Evidence: backend/modules/student_teams/weekly_progress/views.py:15, :71, :111, :173. Confidence: direct static control-flow finding. Proposed direction: one scoped queryset and explicit active Capstone context for all operations.

## A03 - High: stage configuration can rewrite pending grades in another semester

StageGradingConfig.save filters pending grades using Q(defense_stage=self.defense_stage) OR a semester-scoped legacy-label match. The first branch lacks semester=self.semester, so pending grades for the same stage in older/other terms can receive the edited weights. Impact: a term-specific configuration change can alter another term's pending academic data.

Evidence: backend/modules/defense/stages/models.py:225 (save method/query). Confidence: direct static query finding; no cross-term write was attempted. Proposed direction: constrain the complete update queryset to the configuration semester, then handle legacy identity inside that scope.

## A04 - Medium: verdict-based completion and archive-queue passing rules diverge

TeamGrade.result and passed_and_cleared accept an Approved verdict independently of the numeric 75 threshold. finalize_for_archive only applies its numeric threshold check when there is no verdict. In contrast, project_archive._complete_passing_grades unconditionally requires final_grade >= 75 for Capstone. Impact: an approved, finalized team can be passed in core progression but absent from a legacy archive queue. The shared repository has a separate accepted-file visibility path, so the discrepancy does not prove every archive screen is blocked.

Evidence: grading/grades/models.py:result; defense_workflow.py:20; services.py:1415; repository/project_archive/services.py:157. REVIEW: decide whether the official verdict, numeric score, or both control each outcome.

## A05 - High: predictable credentials remain in explicit account paths

Admin academic-record rollover's create action uses password=username. AdminResetPasswordView resets password to username. The bulk-user and official-class-list import paths also hash the new username as the initial password; hashing protects storage but does not make that password unpredictable. Impact: an identifier becomes a usable initial/reset password unless the institution requires an additional enforced change. No universal must-change-password mechanism was found in the user model reviewed.

Evidence: user_management/academic_records/views.py:370; user_management/views.py:584, :924, :1248. Confidence: direct code observation. Proposed direction: institution-approved temporary credentials or recovery

flow with enforced reset; never change existing passwords without a separate explicit request.

## A06 - Medium: repository restrictions are matched by IDs across unrelated definitions

Shared archive querysets collect all restricted deliverable_id values from all stages or PIT events and exclude matching IDs globally. Deliverable IDs are unique only within a stage/event. Impact: restricting ID “D1” in one definition can hide a different definition's unrestricted “D1” submission. This is an over-restriction/data-context problem; it is not evidence of a restricted-file leak.

Evidence: repository/archive/services.py:pit_visible_deliverables_queryset, capstone_visible_queryset, restricted queryset helpers; defense/stages/models.py and defense/scheduler/models.py deliverable uniqueness. Proposed direction: match restriction using the owning stage/event as well as ID and applicable historical identity.

## A07 - High: admin rollover advances Capstone teams beyond the submitted cohort

RolloverConfirmView accumulates affected_student_ids but then calls _advance_capstone_teams(active), which loops over every Capstone team. It moves qualifying third-year second-semester teams to the active school year's first-semester record and also moves fourth-year second-semester teams into an extended phase. The helper does not filter by selected students, expected source school year, or completion/outcome.

Evidence: user_management/academic_records/views.py:322, :415, :431. Impact: confirming a limited student rollover can move unrelated/historical Capstone teams. Confidence: direct static helper analysis. Proposed direction: explicit source/target term and selected team set with preview/confirmation results.

## A08 - Medium: required minutes are displayed but absent from the post-file progression gate

minutes_definition returns a system requirement and stage_minutes_requirement reports completion from the current signed record. post_deliverables_complete selects only required definitions whose type is post; generated minutes use type system. Impact: the stage may progress through that gate without completed signatures even when minutes are displayed as required. This may be intentional separation rather than a defect.

Evidence: defense/minutes/requirements.py:minutes_definition and stage_minutes_requirement; repository/deliverables/services.py:507; student_teams/services.py:stage_progression_blocker and mark_stage_result. REVIEW: determine whether required minutes must block publication, progression, official group completion, or only compliance closure.

## A09 - Medium: multi-role visibility varies across feature paths

visible_schedules_for unions PIT lead scope with adviser/panelist/documenter assignments. visible_teams_for and grade_records_for return early for PIT leads, so the same faculty's independently assigned Capstone teams/grades can be absent from those generic paths. Deliverable team scope unions faculty responsibilities instead. Impact: a multi-role faculty account may see its schedule/deliverables while losing some generic team/grade listings.

Evidence: authentication_access_control/scopes.py:57, :100, :181; repository/deliverables/services.py:team_queryset_for_user. Confidence: static scope comparison; specific workspace behavior needs a multi-role click-through. REVIEW: independent responsibilities should remain usable while management stays PIT-scoped, if that is the intended policy.

### A10 - Review: configuration/history, security defaults, and integration edges

Capstone stage deliverables are global rather than snapshotted by term, so editing a checklist can change how history is interpreted. PIT event names are semester-unique despite year ownership. DEBUG defaults to true if deployment fails to override it, although production security headers/throttles and production WebSocket origin validation now exist. Realtime uses a JWT query parameter. The production ASGI comment discusses allowing mobile clients without Origin, but the implementation wraps the normal AllowedHostsOriginValidator; mobile production behavior needs a dedicated test. Notifications can fail independently of the core action. No production claim is made about TLS, Redis, SMTP, SMS delivery, backups, file persistence, worker topology, or load behavior.

Evidence: defense/stages/models.py; scheduler/models.py; defensys_backend/settings.py:56 and asgi.py; realtime/consumers.py; notifications/services.py. These are follow-up validation items, not proof of an active production incident.

### A11 - Review: legacy routes and labels can misrepresent supported behavior

Existing flow documents mention older apps, student mobile-only access, fixed stage assumptions, and an older endpoint count. Current code has 181 resolved API patterns and a unified archive/project-library structure. Legacy /repository/audit/ aliases management endpoints; upload-pit/upload-capstone exist while upload permission is disabled. Uploader remains a working general-document path. A registered route, an old provider method, and a supported user flow are different things.

Evidence: backend/defensys_backend/urls.py plus resolved appendix; repository/urls.py; project_archive/services.py:repository_scope; frontend/navigation/workspace_access.dart; frontend/web/uploader/uploader_dashboard.dart. Proposed direction: approve the intended map first, then update canonical documentation and retire obsolete surfaces deliberately.

## 8. Owner verification checklist

For each line mark Correct / Change / Unsure and add the intended rule. These are decisions about workflow behavior, not requests to run system actions.

V01. Students can use the student workspace on both web and mobile; administrative work remains web-oriented.

V02. Faculty can hold several responsibilities, and independent adviser/panel/documenter work should remain accessible alongside PIT management.

V03. Exactly one semester is active; switching terms does not itself roll students or teams forward.

V04. Capstone 1 intake normally occurs in third-year second semester; Capstone 2 continuation normally occurs in fourth-year first semester.

V05. New Capstone teams may also be created during continuation when the admin enables team creation.

V06. PIT leads manage only their assigned year; section instructors manage assigned term/year/sections.

V07. PIT cohort rollover advances academic records and preserves section; teams are regrouped separately and PIT scores do not decide academic promotion.

V08. Capstone stages are configurable and ordered; name the exact intended stages and whether any are optional.

V09. Before-endorsement requirements and before-defense presentation materials are distinct timing groups.

V10. Capstone endorsement belongs to the assigned adviser/admin; define which PIT faculty can endorse which teams.

V11. Scheduling requires earlier stages to be passed/cleared and required post files accepted.

V12. Panel evaluation opens on the defense day and remains available after the clock slot until the workflow closes it.

V13. Capstone peer evaluation waits for all required institutional/external panelists; PIT peer evaluation uses an explicit event switch.

V14. Each student evaluates every other member, excluding self; define policy for one-person teams and permitted administrative overrides.

V15. Adviser grading requires an endorsed/scheduled Capstone context, rather than opening immediately at team creation.

V16. Rubrics support team, individual, and mixed criteria; weights vary by stage/event, with defaults 50/30/20 and 80/20.

V17. The official Capstone verdict can differ from a numeric score; specify whether numeric 75 must also be required for progression/archive.

V18. Only chair/admin issues an official Capstone verdict; PIT does not use those verdicts.

V19. Chair, assigned adviser, or admin can clear revisions; adviser/admin verifies re-defense readiness when required.

V20. Failed/rejected concepts need an explicit admin recovery decision; replacement concept starts a new project version and retains old evidence.

V21. Required minutes need documenter → adviser → chair signatures; specify the exact point where missing minutes must block the workflow.

V22. Official group completion, per-team publication, stage passing, and repository file visibility are separate states.

V23. Shared repository includes accepted nonrestricted post files for authenticated users; specify whether any content should be anonymous/public internet access.

V24. Uploader handles general team documents; authoritative stage archive material comes through the deliverable approval flow.

V25. Accepted archive replacement requires a reason, explicit reopen/replacement, review and retained evidence.

V26. Protected grade amendments need explicit approval/acknowledgment; decide whether the approver must be a different admin from the requester.

V27. Historical terms/projects should be read-only except explicit authorized correction/replacement workflows.

V28. Analytics and reports summarize system evidence; they do not automatically approve a curriculum or decide a defense outcome.

V29. Confirm the account onboarding/reset policy; username-as-password should not be assumed to be an approved long-term rule.

V30. Confirm whether PIT event names may repeat independently across year levels and whether stage requirements must be snapshotted per semester.

# 9. Verification performed and limits

Static inventory parsed the backend modules without syntax errors and resolved the live Django URL declarations without running migrations or querying/modifying application records. It counted 49 model classes, 181 registered API patterns (including aliases), 68 backend test files with 1,190 statically discovered test methods, and 126 frontend service/navigation/notification files. These inventory numbers are structural counts, not a claim that all tests or all HTTP operations were exercised.

Focused backend verification: 168 tests passed in 124.255 seconds, system check reported no issues. The suites covered scheduler team readiness, year scoping, session planning, operations, stage setup readiness, Capstone workflow/recovery, grading availability, review-and-endorse, workflow display status, and stage-file access. These ran through manage.py test --keepdb --noinput on the separate test database. Intentional error/denial tests emit

warning/error logs; the final suite result was OK. Tests were not run against defensys_db and no real account credentials were modified.

Focused frontend verification: 61 tests passed across workspace routing, workflow-status parsing, review-and-endorse provider/UI, PIT event management, PIT instructor workspace, and adviser workspace. The initial sandboxed launcher could not open the Flutter SDK lockfile outside the repository; the same selected tests passed after that SDK access was allowed. This was not the entire Flutter test suite or a real-device browser walkthrough.

The audit did not send email/SMS to real recipients, invoke the automation scripts against the live API, seed demo data, change business rules, update existing directives, or deploy anything. Existing uncommitted implementation changes were preserved. The added read-only execution/audit_system_inventory.py makes inventory regeneration repeatable; regenerable audit sources are under .tmp/system-flow-audit/.

Next verification order: approve the main PIT/Capstone flow and V01-V30 → resolve the high-severity access/data-scope findings → reconcile completion/archive/minutes policy → run focused regressions for approved fixes → perform role-based browser/device validation with representative multi-role and historical records.

# 10. Technical inventory

The following appendices were generated from code to prevent endpoint/model omissions. Permissions listed on a view are only the first layer; method checks, assignment checks, service scopes, active-term rules and immutable-history checks still apply. Compatibility aliases are intentionally retained in the endpoint list.

## Appendix A. Resolved API catalog

Methods are resolved from class inheritance; aliases and dynamically expanded catalog reports are included. Service checks may disable or further restrict an operation.

GET /api/health/ → defensys_backend.views.HealthCheckView

POST /api/login/ → authentication_access_control.views.CustomTokenObtainPairView

POST /api/token/refresh/ → authentication_access_control.views.ThrottledTokenRefreshView

POST /api/logout/ → authentication_access_control.views.LogoutView

GET,PATCH /api/me/ → authentication_access_control.views.CurrentUserView

GET /api/me/history/ → authentication_access_control.views.UserHistoryView

POST /api/change-password/ → authentication_access_control.views.ChangePasswordView

GET /api/audit-logs/ → authentication_access_control.views.SystemAuditLogListView

PATCH /api/audit-logs/<int:pk>/review/ → authentication_access_control.views.SystemAuditLogReviewView

GET /api/dashboards/schedule-planner/ → dashboards.views.SchedulePlannerView

GET /api/dashboards/admin/ → dashboards.views.AdminDashboardView

GET /api/dashboards/pit-lead/cohort/ → dashboards.views.PitLeadCohortView

GET /api/dashboards/pit-lead/cohort/rollover-preview/ → dashboards.views.PitLeadCohortRolloverPreviewView

POST /api/dashboards/pit-lead/cohort/rollover/ → dashboards.views.PitLeadCohortRolloverConfirmView

GET /api/dashboards/faculty/ → dashboards.views.FacultyDashboardView

GET /api/dashboards/student/ → dashboards.views.StudentDashboardView

GET /api/dashboards/panelist/ → dashboards.views.PanelistDashboardView

GET,POST /api/academic-periods/ → academic_period_management.views.AcademicPeriodListCreateView

PATCH,DELETE /api/academic-periods/<int:school_year_id>/ → academic_period_management.views.SchoolYearDetailView

POST /api/academic-periods/<int:school_year_id>/semesters/ → academic_period_management.views.SemesterCreateView

GET /api/academic-periods/semesters/<int:semester_id>/transition-preview/ → academic_period_management.views.SemesterTransitionPreviewView

POST /api/academic-periods/semesters/<int:semester_id>/activate/ → academic_period_management.views.SemesterActivateView

PATCH,DELETE /api/academic-periods/semesters/<int:semester_id>/ → academic_period_management.views.SemesterStatusView

GET,POST /api/users/external-evaluators/ → user_management.external_evaluator_views.ExternalEvaluatorsView

GET,PATCH /api/users/external-evaluators/<int:evaluator_id>/ → user_management.external_evaluator_views.ExternalEvaluatorDetailView

POST /api/users/external-invitations/ → user_management.external_evaluator_views.ExternalInvitationsView

PATCH /api/users/external-invitations/<int:invitation_id>/ → user_management.external_evaluator_views.ExternalInvitationDetailView

GET,POST /api/users/panelist-requests/ → user_management.panelist_eligibility_views.PanelistEligibilityRequestsView

GET,PATCH /api/users/panelist-requests/<int:request_id>/ → user_management.panelist_eligibility_views.PanelistEligibilityRequestReviewView

GET,POST /api/users/academic-records/ → user_management.academic_records.views.StudentAcademicRecordListCreateView

GET,POST /api/users/academic-records/rollover-preview/ → user_management.academic_records.views.RolloverPreviewView

POST /api/users/academic-records/rollover/ → user_management.academic_records.views.RolloverConfirmView

PATCH,DELETE /api/users/academic-records/<int:record_id>/ → user_management.academic_records.views.StudentAcademicRecordDetailView

GET,POST /api/users/ → user_management.views.UserListCreateView

POST /api/users/bulk-import/ → user_management.views.BulkImportUsersView

POST /api/users/pit-lead/student-import/ → user_management.views.PitLeadStudentImportView

POST /api/users/pit-lead/official-class-list-import/ → user_management.views.PitLeadOfficialClassListImportView

GET,POST /api/users/pit-instructors/ → user_management.views.SectionInstructorAssignmentView

PATCH /api/users/pit-instructors/<int:assignment_id>/ → user_management.views.SectionInstructorAssignmentDetailView

GET,POST /api/users/guest-codes/ → user_management.views.GuestPanelistCodeListCreateView

PATCH /api/users/guest-codes/<int:code_id>/ → user_management.views.GuestPanelistCodeDetailView

POST /api/users/guest-codes/exchange/ → user_management.views.GuestCodeExchangeView

GET /api/users/guest-codes/validate/<str:code>/ → user_management.views.GuestCodeValidateView

POST,DELETE /api/users/e-signature/ → user_management.views.UserESignatureView

GET /api/users/<int:user_id>/adviser-assignments/ → user_management.views.UserAdviserAssignmentHistoryView

GET /api/users/<int:user_id>/role-assignments/ → user_management.views.UserRoleAssignmentHistoryView

POST /api/users/<int:user_id>/reset-password/ → user_management.views.AdminResetPasswordView

GET,PATCH,DELETE /api/users/<int:user_id>/ → user_management.views.UserDetailView

GET /api/teams/documents/ → student_teams.documents.views.TeamDocumentListView

POST /api/teams/documents/upload/ → student_teams.documents.views.TeamDocumentUploadView

GET,DELETE /api/teams/documents/<int:document_id>/ → student_teams.documents.views.TeamDocumentDetailView

GET /api/teams/documents/<int:document_id>/download/ → student_teams.documents.views.TeamDocumentDownloadView

GET,POST /api/teams/weekly-progress/ → student_teams.weekly_progress.views.StudentWeeklyProgressListCreateView

GET,PUT,DELETE /api/teams/weekly-progress/<int:pk>/ → student_teams.weekly_progress.views.StudentWeeklyProgressDetailView

GET /api/teams/weekly-progress/<int:pk>/file/ → student_teams.weekly_progress.views.WeeklyProgressReportFileView

GET,POST /api/teams/section-assignments/ → student_teams.views.SectionAssignmentListCreateView

GET,PATCH,DELETE /api/teams/section-assignments/<int:pk>/ → student_teams.views.SectionAssignmentDetailView

GET,POST /api/teams/ → student_teams.views.StudentTeamListCreateView

POST /api/teams/bulk-import/preview/ → student_teams.views.BulkImportTeamsPreviewView

POST /api/teams/bulk-import/ → student_teams.views.BulkImportTeamsView

POST /api/teams/<int:team_id>/remind/ → student_teams.views.StudentTeamSendReminderView

GET /api/teams/<int:team_id>/adviser-history/ → student_teams.views.TeamAdviserHistoryView

GET,PATCH /api/teams/<int:team_id>/section/ → student_teams.views.SectionDetailsView

GET,PATCH,DELETE /api/teams/<int:team_id>/ → student_teams.views.StudentTeamDetailView

GET,POST /api/defense/stages/ → defense.stages.views.DefenseStageListCreateView

POST /api/defense/stages/reorder/ → defense.stages.views.DefenseStageReorderView

GET,PATCH,DELETE /api/defense/stages/<int:stage_id>/ → defense.stages.views.DefenseStageDetailView

GET,PATCH /api/defense/stages/<int:stage_id>/grading-config/ → defense.stages.views.StageGradingConfigView

POST /api/defense/stages/<int:stage_id>/deliverables/ → defense.stages.views.StageDeliverableListCreateView

PATCH,DELETE /api/defense/stages/<int:stage_id>/deliverables/<int:deliverable_id>/ → defense.stages.views.StageDeliverableDetailView

GET,POST /api/defense/schedules/ → defense.scheduler.views.DefenseScheduleListCreateView

GET,POST,DELETE /api/defense/schedules/pit-event-config/ → defense.scheduler.views.PitEventConfigLookupView

POST /api/defense/schedules/generate-plan/ → defense.scheduler.views.DefenseScheduleGeneratePlanView

POST /api/defense/schedules/confirm-plan/ → defense.scheduler.views.DefenseScheduleConfirmPlanView

GET /api/defense/schedules/panelist-assignments/ → defense.scheduler.views.PanelistAssignmentsView

GET /api/defense/schedules/panelist-results/ → defense.scheduler.views.PanelistResultsView

GET /api/defense/schedules/guest-assignments/ → defense.scheduler.views.GuestPanelistAssignmentsView

GET /api/defense/schedules/guest-panelist-results/ → defense.scheduler.views.GuestPanelistResultsView

POST /api/defense/schedules/submit-grades/ → defense.scheduler.views.PanelistGradeSubmissionView

POST /api/defense/schedules/grade-draft/ → defense.scheduler.views.PanelistEvaluationDraftView

POST /api/defense/schedules/guest-grade-draft/ → defense.scheduler.views.GuestPanelistEvaluationDraftView

POST /api/defense/schedules/guest-submit-grades/ → defense.scheduler.views.GuestPanelistGradeSubmissionView

PATCH,DELETE /api/defense/schedules/<int:schedule_id>/ → defense.scheduler.views.DefenseScheduleDetailView

POST /api/defense/schedules/<int:schedule_id>/material-deadline/ → defense.scheduler.views.DefenseMaterialDeadlineView

PATCH /api/defense/schedules/<int:schedule_id>/verdict/ → defense.scheduler.views.DefenseScheduleVerdictView

GET /api/defense/board/ → defense.board.views.DefenseBoardListView

GET,POST /api/defense/board/operations/ → defense.board.views.DefenseBoardOperationsView

PATCH,DELETE /api/defense/board/<int:schedule_id>/ → defense.board.views.DefenseBoardDetailView

GET,PATCH /api/defense/minutes/documenter-pool/ → defense.minutes.documenter_pool.DocumenterPoolView

GET /api/defense/minutes/my-assignments/ → defense.minutes.views.MyDocumenterAssignmentsView

GET,PATCH /api/defense/minutes/<int:schedule_id>/ → defense.minutes.views.MinutesDetailView

POST /api/defense/minutes/<int:schedule_id>/submit/ → defense.minutes.views.MinutesSubmitView

POST /api/defense/minutes/<int:schedule_id>/sign-adviser/ → defense.minutes.views.MinutesSignAdviserView

POST /api/defense/minutes/<int:schedule_id>/sign-chairman/ → defense.minutes.views.MinutesSignChairmanView

GET /api/defense/minutes/<int:schedule_id>/preview/ → defense.minutes.views.MinutesPreviewView

GET /api/defense/minutes/<int:schedule_id>/pdf/ → defense.minutes.views.MinutesPdfView

GET,POST /api/grading/rubrics/ → grading.rubrics.views.RubricListCreateView

GET,PATCH,DELETE /api/grading/rubrics/<int:rubric_id>/ → grading.rubrics.views.RubricDetailView

POST /api/grading/rubrics/<int:rubric_id>/publish/ → grading.rubrics.views.RubricPublishView

PATCH /api/grading/rubrics/<int:rubric_id>/weights/ → grading.rubrics.views.RubricWeightsView

GET /api/grading/grades/ → grading.grades.views.GradeCenterListView

POST /api/grading/grades/sync/ → grading.grades.views.GradeCenterSyncView

PATCH /api/grading/grades/evaluation-settings/ → grading.grades.views.CapstoneEvaluationSettingsView

GET,PATCH /api/grading/grades/group-settings/ → grading.grades.views.GradeCenterGroupSettingsView

GET,PATCH /api/grading/grades/<int:grade_id>/ → grading.grades.views.GradeCenterDetailView

GET,POST /api/grading/grades/<int:grade_id>/corrections/ → grading.grades.views.GradeCorrectionsView

POST /api/grading/grades/<int:grade_id>/publish/ → grading.grades.views.GradeCenterPublishView

PATCH /api/grading/grades/<int:grade_id>/verdict/ → grading.grades.views.TeamGradeVerdictView

GET,POST /api/grading/grades/<int:grade_id>/workflow/ → grading.grades.views.TeamGradeWorkflowView

GET /api/grading/grades/adviser-grades/ → grading.grades.adviser_views.AdviserGradeListView

POST /api/grading/grades/adviser-grades/<int:grade_id>/submit/ → grading.grades.adviser_views.AdviserSubmitGradeView

POST /api/grading/grades/peer-evaluations/ → grading.grades.peer_views.StudentPeerEvaluationSubmitView

GET /api/repository/archive/ → repository.archive.views.ProjectArchiveListView

GET /api/repository/archive/search/ → repository.archive.views.ProjectArchiveSearchView

GET,POST,DELETE /api/repository/archive/reviews/ → repository.archive.views.RepositoryReviewView

GET,POST,DELETE /api/repository/archive/shelf/ → repository.archive.views.UserBookShelfView

GET /api/repository/deliverables/ → repository.deliverables.views.CapstoneDeliverablesListView

POST /api/repository/deliverables/upload/ → repository.deliverables.views.CapstoneDeliverableUploadView

POST /api/repository/deliverables/remove/ → repository.deliverables.views.CapstoneDeliverableRemoveView

POST /api/repository/deliverables/endorse/ → repository.deliverables.views.CapstoneDeliverableEndorseView

POST /api/repository/deliverables/review-endorse/ → repository.deliverables.views.ReviewEndorseView

POST /api/repository/deliverables/unendorse/ → repository.deliverables.views.CapstoneDeliverableUnendorseView

POST /api/repository/deliverables/review/ → repository.deliverables.views.CapstoneDeliverableReviewView

POST /api/repository/deliverables/unlock/ → repository.deliverables.views.CapstoneDeliverableUnlockView

GET /api/repository/deliverables/stage-access/ → repository.deliverables.views.StageFileAccessView

POST /api/repository/deliverables/compile-weekly-reports/ → repository.deliverables.views.CompileWeeklyReportsView

GET /api/repository/project-archive/ → repository.project_archive.views.ProjectArchiveListView

POST /api/repository/project-archive/upload-pit/ → repository.project_archive.views.ProjectArchiveUploadPitView

POST /api/repository/project-archive/upload-capstone/ → repository.project_archive.views.ProjectArchiveUploadCapstoneView

POST /api/repository/project-archive/request-resubmission/ → repository.project_archive.views.ProjectArchiveRequestResubmissionView

POST /api/repository/project-archive/cancel-file-replacement/ → repository.project_archive.views.ProjectArchiveCancelReplacementView

POST /api/repository/project-archive/override-status/ → repository.project_archive.views.ProjectArchiveOverrideStatusView

POST /api/repository/project-archive/replace-file/ → repository.project_archive.views.ProjectArchiveReplaceFileView

GET /api/repository/project-archive/trail/ → repository.project_archive.views.ProjectArchiveTrailView

GET /api/repository/project-archive/export/ → repository.project_archive.views.ProjectArchiveExportView

GET /api/repository/audit/ → repository.project_archive.views.ProjectArchiveListView

POST /api/repository/audit/upload-pit/ → repository.project_archive.views.ProjectArchiveUploadPitView

POST /api/repository/audit/upload-capstone/ → repository.project_archive.views.ProjectArchiveUploadCapstoneView

POST /api/repository/audit/request-resubmission/ → repository.project_archive.views.ProjectArchiveRequestResubmissionView

POST /api/repository/audit/cancel-file-replacement/ → repository.project_archive.views.ProjectArchiveCancelReplacementView

POST /api/repository/audit/override-status/ → repository.project_archive.views.ProjectArchiveOverrideStatusView

POST /api/repository/audit/replace-file/ → repository.project_archive.views.ProjectArchiveReplaceFileView

GET /api/repository/audit/trail/ → repository.project_archive.views.ProjectArchiveTrailView

GET /api/repository/audit/export/ → repository.project_archive.views.ProjectArchiveExportView

GET /api/curriculum-analytics/explorer/ → curriculum_analytics.views.CurriculumExplorerView

GET /api/curriculum-analytics/explorer/report/ → curriculum_analytics.explorer_report.CurriculumExplorerReportView

GET /api/curriculum-analytics/explorer/projects/<str:project_id>/ → curriculum_analytics.views.CurriculumExplorerDetailView

GET /api/curriculum-analytics/ → curriculum_analytics.views.CurriculumAnalyticsView

GET,POST /api/curriculum-analytics/proposal/ → curriculum_analytics.views.CurriculumProposalView

GET /api/curriculum-analytics/proposal/pdf/ → curriculum_analytics.views.CurriculumProposalPdfView

GET /api/reports/faculty-teams-overview/ → reports.faculty_reports.FacultyTeamsOverviewView

GET /api/reports/faculty-team/<int:team_id>/ → reports.faculty_reports.FacultyTeamSummaryView

GET /api/reports/catalog-options/ → reports.catalog_reports.CatalogOptionsView

GET /api/reports/academic-periods/ → reports.catalog_reports.CatalogReportView

GET /api/reports/student-enrollment/ → reports.catalog_reports.CatalogReportView

GET /api/reports/faculty-directory/ → reports.catalog_reports.CatalogReportView

GET /api/reports/rubrics/ → reports.catalog_reports.CatalogReportView

GET /api/reports/project-records/ → reports.catalog_reports.CatalogReportView

GET /api/reports/external-evaluators/ → reports.catalog_reports.CatalogReportView

GET /api/reports/defense-stage-setup/ → reports.catalog_reports.CatalogReportView

GET /api/reports/pit-event-setup/ → reports.catalog_reports.CatalogReportView

GET /api/reports/readiness/ → reports.catalog_reports.CatalogReportView

GET /api/reports/overview/ → reports.catalog_reports.CatalogReportView

GET /api/reports/curriculum-summary/ → reports.catalog_reports.CatalogReportView

GET /api/reports/rubrics/<int:rubric_id>/ → reports.catalog_reports.CatalogReportView

GET /api/reports/assessment-candidates/ → reports.views.AssessmentCandidatesView

GET /api/reports/team-grade/<int:team_id>/ → reports.views.TeamGradeReportView

GET /api/reports/individual-grade/<int:student_id>/ → reports.views.IndividualGradeReportView

GET /api/reports/semester-grades/ → reports.views.SemesterGradesReportView

GET /api/reports/defense-schedules/ → reports.views.DefenseScheduleReportView

GET /api/reports/team-roster/ → reports.views.TeamRosterReportView

GET /api/reports/user-directory/ → reports.views.UserDirectoryReportView

GET /api/reports/audit-trail/ → reports.views.AuditTrailReportView

GET /api/notifications/ → notifications.views.NotificationListView

GET,POST /api/notifications/<int:pk>/read/ → notifications.views.NotificationReadView

GET,POST /api/notifications/<int:pk>/ → notifications.views.NotificationReadView

POST /api/notifications/read-all/ → notifications.views.NotificationReadAllView

GET /api/media/files/<path:file_path> → defensys_backend.media_views.AuthenticatedMediaFileView

POST /api/password-reset/ → authentication_access_control.password_reset.RequestPasswordResetView

POST /api/password-reset/verify-otp/ → authentication_access_control.password_reset.VerifyPasswordResetOTPView

POST /api/password-reset/confirm/ → authentication_access_control.password_reset.ConfirmPasswordResetAPIView

## Appendix B. All model classes and their relationship fields

Only explicit Django model classes and relationship declarations are listed; inherited auth tables, generated join tables and scalar fields are not counted as separate feature models.

SchoolYear - backend/modules/academic_period_management/models.py:20

Semester - backend/modules/academic_period_management/models.py:31

school_year: ForeignKey → SchoolYear

SemesterTransitionLog - backend/modules/academic_period_management/models.py:101

from_semester: ForeignKey → Semester

to_semester: ForeignKey → Semester

changed_by: ForeignKey → settings.AUTH_USER_MODEL

User - backend/modules/authentication_access_control/models.py:14

SystemAuditLog - backend/modules/authentication_access_control/models.py:57

actor: ForeignKey → settings.AUTH_USER_MODEL

reviewed_by: ForeignKey → settings.AUTH_USER_MODEL

SystemAuditReview - backend/modules/authentication_access_control/models.py:135

audit_log: ForeignKey → SystemAuditLog

reviewer: ForeignKey → settings.AUTH_USER_MODEL

PasswordResetOTP - backend/modules/authentication_access_control/models.py:155

user: ForeignKey → settings.AUTH_USER_MODEL

DefenseMinutes - backend/modules/defense/minutes/models.py:4

schedule: OneToOneField → 'defense.DefenseSchedule'

documenter_signed_by: ForeignKey → settings.AUTH_USER_MODEL

adviser_signed_by: ForeignKey → settings.AUTH_USER_MODEL

chairman_signed_by: ForeignKey → settings.AUTH_USER_MODEL

MinutesPanelistComment - backend/modules/defense/minutes/models.py:85

minutes: ForeignKey → DefenseMinutes

panelist: ForeignKey → settings.AUTH_USER_MODEL

guest_invitation: ForeignKey → 'user_management.GuestPanelistCode'

DefenseMinutesRevision - backend/modules/defense/minutes/models.py:125

minutes: ForeignKey → DefenseMinutes

actor: ForeignKey → settings.AUTH_USER_MODEL

PanelistEvaluationDraft - backend/modules/defense/scheduler/models.py:9

schedule: ForeignKey → 'defense.DefenseSchedule'

panelist: ForeignKey → settings.AUTH_USER_MODEL

DefenseSchedule - backend/modules/defense/scheduler/models.py:44

semester: ForeignKey → 'academic_period_management.Semester'

team: ForeignKey → 'student_teams.StudentTeam'

defense_stage: ForeignKey → 'defense.DefenseStage'

rubric: ForeignKey → 'grading.Rubric'

created_by: ForeignKey → settings.AUTH_USER_MODEL

documenter: ForeignKey → settings.AUTH_USER_MODEL

panelists: ManyToManyField → settings.AUTH_USER_MODEL

SchedulePanelist - backend/modules/defense/scheduler/models.py:317

schedule: ForeignKey → DefenseSchedule

panelist: ForeignKey → settings.AUTH_USER_MODEL

PitEventGradingConfig - backend/modules/defense/scheduler/models.py:368

semester: ForeignKey → 'academic_period_management.Semester'

panel_rubric: ForeignKey → 'grading.Rubric'

peer_rubric: ForeignKey → 'grading.Rubric'

PitEventDeliverable - backend/modules/defense/scheduler/models.py:506

pit_event_config: ForeignKey → PitEventGradingConfig

StageDeliverable - backend/modules/defense/stages/models.py:21

defense_stage: ForeignKey → 'DefenseStage'

DefenseStage - backend/modules/defense/stages/models.py:120

StageGradingConfig - backend/modules/defense/stages/models.py:162

defense_stage: ForeignKey → DefenseStage

semester: ForeignKey → 'academic_period_management.Semester'

panel_rubric: ForeignKey → 'grading.Rubric'

adviser_rubric: ForeignKey → 'grading.Rubric'

peer_rubric: ForeignKey → 'grading.Rubric'

GradeCorrection - backend/modules/grading/grades/correction_models.py:5

grade: ForeignKey → 'grading.TeamGrade'

requested_by: ForeignKey → settings.AUTH_USER_MODEL

approved_by: ForeignKey → settings.AUTH_USER_MODEL

TeamGrade - backend/modules/grading/grades/models.py:45

schedule: ForeignKey → 'defense.DefenseSchedule'

team: ForeignKey → 'student_teams.StudentTeam'

semester: ForeignKey → 'academic_period_management.Semester'

defense_stage: ForeignKey → 'defense.DefenseStage'

pit_event_config: ForeignKey → 'defense.PitEventGradingConfig'

verdict_by: ForeignKey → settings.AUTH_USER_MODEL

revisions_cleared_by: ForeignKey → settings.AUTH_USER_MODEL

redefense_verified_by: ForeignKey → settings.AUTH_USER_MODEL

published_by: ForeignKey → settings.AUTH_USER_MODEL

GradeBreakdown - backend/modules/grading/grades/models.py:336

source_submission: ForeignKey → 'grading.PanelistGradeSubmission'

team_grade: ForeignKey → TeamGrade

student: ForeignKey → settings.AUTH_USER_MODEL

rubric: ForeignKey → 'grading.Rubric'

PanelistGradeSubmission - backend/modules/grading/grades/models.py:401

team_grade: ForeignKey → TeamGrade

schedule: ForeignKey → 'defense.DefenseSchedule'

panelist: ForeignKey → settings.AUTH_USER_MODEL

student: ForeignKey → settings.AUTH_USER_MODEL

PanelistCriterionScore - backend/modules/grading/grades/models.py:485

submission: ForeignKey → PanelistGradeSubmission

criterion: ForeignKey → 'grading.RubricCriterion'

StudentStageGrade - backend/modules/grading/grades/models.py:538

team_grade: ForeignKey → TeamGrade

student: ForeignKey → settings.AUTH_USER_MODEL

PeerEvaluationSubmission - backend/modules/grading/grades/models.py:579

team_grade: ForeignKey → TeamGrade

evaluator: ForeignKey → settings.AUTH_USER_MODEL

evaluatee: ForeignKey → settings.AUTH_USER_MODEL

GradeAttemptHistory - backend/modules/grading/grades/models.py:625

team_grade: ForeignKey → TeamGrade

schedule: ForeignKey → 'defense.DefenseSchedule'

verdict_by: ForeignKey → settings.AUTH_USER_MODEL

snapshotted_by: ForeignKey → settings.AUTH_USER_MODEL

Rubric - backend/modules/grading/rubrics/models.py:6

semester: ForeignKey → 'academic_period_management.Semester'

defense_stage: ForeignKey → 'defense.DefenseStage'

created_by: ForeignKey → settings.AUTH_USER_MODEL

RubricCriterion - backend/modules/grading/rubrics/models.py:140

rubric: ForeignKey → Rubric

Notification - backend/modules/notifications/models.py:33

recipient: ForeignKey → settings.AUTH_USER_MODEL

sender: ForeignKey → settings.AUTH_USER_MODEL

ArchiveEntry - backend/modules/repository/archive/models.py:22

team: ForeignKey → 'student_teams.StudentTeam'

defense_stage: ForeignKey → 'defense.DefenseStage'

pit_event_config: ForeignKey → 'defense.PitEventGradingConfig'

uploaded_by: ForeignKey → settings.AUTH_USER_MODEL

RepositoryReview - backend/modules/repository/archive/models.py:198

user: ForeignKey → settings.AUTH_USER_MODEL

UserBookShelf - backend/modules/repository/archive/models.py:238

user: ForeignKey → settings.AUTH_USER_MODEL

DeliverableSubmission - backend/modules/repository/deliverables/models.py:10

team: ForeignKey → 'student_teams.StudentTeam'

archive_semester: ForeignKey → 'academic_period_management.Semester'

pit_event_config: ForeignKey → 'defense.PitEventGradingConfig'

reviewed_by: ForeignKey → settings.AUTH_USER_MODEL

uploaded_by: ForeignKey → settings.AUTH_USER_MODEL

DeliverableSubmissionFile - backend/modules/repository/deliverables/models.py:227

submission: ForeignKey → 'DeliverableSubmission'

ProjectArchiveLog - backend/modules/repository/project_archive/models.py:5

actor: ForeignKey → settings.AUTH_USER_MODEL

TeamDocument - backend/modules/student_teams/documents/models.py:5

team: ForeignKey → 'student_teams.StudentTeam'

uploaded_by: ForeignKey → settings.AUTH_USER_MODEL

StudentTeam - backend/modules/student_teams/models.py:10

semester: ForeignKey → 'academic_period_management.Semester'

leader: ForeignKey → settings.AUTH_USER_MODEL

adviser: ForeignKey → settings.AUTH_USER_MODEL

members: ManyToManyField → settings.AUTH_USER_MODEL

TeamMembership - backend/modules/student_teams/models.py:118

team: ForeignKey → StudentTeam

student: ForeignKey → settings.AUTH_USER_MODEL

SectionAssignment - backend/modules/student_teams/models.py:143

semester: ForeignKey → 'academic_period_management.Semester'

project_manager: ForeignKey → settings.AUTH_USER_MODEL

TeamStageProgress - backend/modules/student_teams/models.py:180

team: ForeignKey → StudentTeam

semester: ForeignKey → 'academic_period_management.Semester'

defense_stage: ForeignKey → 'defense.DefenseStage'

grade: ForeignKey → 'grading.TeamGrade'

created_by: ForeignKey → settings.AUTH_USER_MODEL

updated_by: ForeignKey → settings.AUTH_USER_MODEL

TeamRecoveryAuthorization - backend/modules/student_teams/models.py:271

team: ForeignKey → StudentTeam

defense_stage: ForeignKey → 'defense.DefenseStage'

semester: ForeignKey → 'academic_period_management.Semester'

authorized_by: ForeignKey → settings.AUTH_USER_MODEL

TeamAdviserAssignment - backend/modules/student_teams/models.py:291

team: ForeignKey → StudentTeam

adviser: ForeignKey → settings.AUTH_USER_MODEL

assigned_by: ForeignKey → settings.AUTH_USER_MODEL

WeeklyProgressReport - backend/modules/student_teams/weekly_progress/models.py:6

student: ForeignKey → settings.AUTH_USER_MODEL

team: ForeignKey → 'student_teams.StudentTeam'

StudentAcademicRecord - backend/modules/user_management/academic_records/models.py:6

student: ForeignKey → settings.AUTH_USER_MODEL

semester: ForeignKey → 'academic_period_management.Semester'

rolled_from: ForeignKey → 'self'

ExternalEvaluator - backend/modules/user_management/models.py:8

created_by: ForeignKey → settings.AUTH_USER_MODEL

reviewed_by: ForeignKey → settings.AUTH_USER_MODEL

GuestPanelistCode - backend/modules/user_management/models.py:35

evaluator: ForeignKey → ExternalEvaluator

schedules: ManyToManyField → 'defense.DefenseSchedule'

defense_schedule: ForeignKey → 'defense.DefenseSchedule'

created_by: ForeignKey → settings.AUTH_USER_MODEL

FacultyRoleAssignment - backend/modules/user_management/models.py:85

user: ForeignKey → settings.AUTH_USER_MODEL

semester: ForeignKey → 'academic_period_management.Semester'

changed_by: ForeignKey → settings.AUTH_USER_MODEL

PanelistEligibilityRequest - backend/modules/user_management/models.py:137

faculty: ForeignKey → settings.AUTH_USER_MODEL

requested_by: ForeignKey → settings.AUTH_USER_MODEL

reviewed_by: ForeignKey → settings.AUTH_USER_MODEL

SectionInstructorAssignment - backend/modules/user_management/models.py:183

faculty: ForeignKey → settings.AUTH_USER_MODEL

semester: ForeignKey → 'academic_period_management.Semester'

assigned_by: ForeignKey → settings.AUTH_USER_MODEL

## Appendix C. Frontend API and navigation wiring

Root provider files often re-export the feature-directory implementation; an export shim is not a separate feature.

frontend/lib/services/academic/academic_period_provider.dart

L60: static String get baseUrl => ApiConfig.academicPeriodsUrl;

frontend/lib/services/academic/curriculum_analytics_provider.dart

L76: static String get baseUrl => ApiConfig.curriculumAnalyticsUrl;

frontend/lib/services/academic/curriculum_explorer_provider.dart

L84: '${ApiConfig.curriculumAnalyticsUrl}/explorer/',

L120: '${ApiConfig.curriculumAnalyticsUrl}/explorer/projects/${Uri.encodeComponent(projectId)}/',

frontend/lib/services/academic/student_academic_records_provider.dart

L86: static final String baseUrl = ApiConfig.studentRecordsUrl;

frontend/lib/services/academic/student_teams_provider.dart

L128: static String get baseUrl => ApiConfig.teamsUrl;

frontend/lib/services/academic/team_detail_provider.dart

L112: Uri.parse('${ApiConfig.teamsUrl}/$_teamId/'),

L181: Uri.parse('${ApiConfig.teamsUrl}/$_teamId/'),

L228: Uri.parse('${ApiConfig.teamsUrl}/$_teamId/adviser-history/'),

L243: Uri.parse('${ApiConfig.teamDocumentsUrl}/?team_id=$_teamId'),

L258: Uri.parse('${ApiConfig.weeklyProgressUrl}/?team_id=$_teamId'),

L276: '${ApiConfig.gradeCenterUrl}/',

L303: final uri = Uri.parse(ApiConfig.capstoneDeliverablesUrl).replace(

frontend/lib/services/admin/external_evaluator_provider.dart

L53: String get _base => ApiConfig.usersUrl;

frontend/lib/services/admin/panelist_requests_provider.dart

L72: Uri.parse('${ApiConfig.usersUrl}/panelist-requests/').replace(

frontend/lib/services/admin/project_archive_provider.dart

L173: static String get baseUrl => ApiConfig.projectArchiveUrl;

frontend/lib/services/admin/reports_provider.dart

L206: '${ApiConfig.baseUrl}/reports/assessment-candidates/',

L232: '${ApiConfig.baseUrl}/reports/$endpoint',

L279: '${ApiConfig.baseUrl}/reports/$endpoint',

frontend/lib/services/admin/repository_provider.dart

L68: static String get baseUrl => ApiConfig.repositoryUrl;

L150: .get(Uri.parse('${ApiConfig.teamDocumentsUrl}/'))

frontend/lib/services/admin/system_audit_provider.dart

L155: '${ApiConfig.baseUrl}/audit-logs/',

L281: final uri = Uri.parse('${ApiConfig.baseUrl}/audit-logs/$logId/review/');

frontend/lib/services/admin/user_management_provider.dart

L74: static final String baseUrl = ApiConfig.usersUrl;

frontend/lib/services/app/dashboard_provider.dart

L50: static String get baseUrl => ApiConfig.dashboardsUrl;

frontend/lib/services/auth/auth_provider.dart

L76: static String get baseUrl => ApiConfig.authUrl;

L153: Uri.parse('${ApiConfig.baseUrl}/users/guest-codes/exchange/'),

frontend/lib/services/defense/capstone_deliverables_provider.dart

L251: static String get baseUrl => ApiConfig.capstoneDeliverablesUrl;

frontend/lib/services/defense/defense_board_provider.dart

L124: static String get baseUrl => ApiConfig.defenseBoardUrl;

L167: Uri.parse('${ApiConfig.defenseSchedulesUrl}/$scheduleId/material-deadline/'),

frontend/lib/services/defense/defense_scheduler_provider.dart

L275: static String get baseUrl => ApiConfig.defenseSchedulesUrl;

L759: '${ApiConfig.usersUrl}/panelist-requests/',

L779: '${ApiConfig.usersUrl}/$facultyId/',

L798: '${ApiConfig.usersUrl}/panelist-requests/$requestId/',

frontend/lib/services/defense/defense_stages_provider.dart

L57: static String get baseUrl => ApiConfig.defenseStagesUrl;

frontend/lib/services/defense/team_defense_provider.dart

L43: ApiConfig.defenseBoardUrl,

frontend/lib/services/grading/adviser_grading_provider.dart

L61: static String get _gradesUrl => '${ApiConfig.gradeCenterUrl}/adviser-grades/';

L138: Uri.parse('${ApiConfig.gradeCenterUrl}/adviser-grades/$gradeId/submit/'),

frontend/lib/services/grading/e_signature_provider.dart

L23: Uri.parse('${ApiConfig.userSignatureUrl}/'),

L49: Uri.parse('${ApiConfig.userSignatureUrl}/'),

frontend/lib/services/grading/grade_center_provider.dart

L152: static String get baseUrl => ApiConfig.gradeCenterUrl;

frontend/lib/services/grading/rubric_engine_provider.dart

L102: static String get baseUrl => ApiConfig.rubricsUrl;

frontend/lib/services/grading/rubrics_provider.dart

L100: static String get baseUrl => ApiConfig.rubricsUrl;

frontend/lib/services/network/authenticated_client.dart

L185: final url = ApiConfig.authenticatedMediaUrl(fileRef);

frontend/lib/services/pit/documenter_provider.dart

L67: static String get minutesUrl => ApiConfig.defenseMinutesUrl;

frontend/lib/services/pit/pit_instructor_provider.dart

L60: static String get baseUrl => '${ApiConfig.usersUrl}/pit-instructors';

frontend/lib/services/pit/pit_lead_cohort_provider.dart

L113: '${ApiConfig.dashboardsUrl}/pit-lead/cohort/',

L176: '${ApiConfig.dashboardsUrl}/pit-lead/cohort/rollover-preview/',

L201: Uri.parse('${ApiConfig.dashboardsUrl}/pit-lead/cohort/rollover/'),

frontend/lib/services/pit/weekly_progress_provider.dart

L48: static String get baseUrl => ApiConfig.weeklyProgressUrl;

frontend/lib/services/realtime/bridge_service.dart

L15: Uri.parse('${ApiConfig.baseUrl}/users/guest-codes/exchange/'),

L36: .get(Uri.parse('${ApiConfig.baseUrl}/users/guest-codes/validate/$code'))

L63: Uri.parse('${ApiConfig.gradeCenterUrl}/peer-evaluations/'),

frontend/lib/services/realtime/realtime_sync_service.dart

L83: final uri = ApiConfig.webSocketGradingUri(token);

frontend/lib/services/repository_review_provider.dart

L44: final response = await client.get(Uri.parse(ApiConfig.repositoryShelfUrl));

L62: final uri = Uri.parse(ApiConfig.repositoryUrl).replace(

L241: ApiConfig.repositoryReviewsUrl,

L260: final uri = Uri.parse(ApiConfig.repositoryReviewsUrl);

L288: final uri = Uri.parse(ApiConfig.repositoryReviewsUrl);

L305: final uri = Uri.parse(ApiConfig.repositoryShelfUrl);

L329: final uri = Uri.parse(ApiConfig.repositoryShelfUrl);

L363: Uri.parse(ApiConfig.repositoryShelfUrl),